MemDB2 Codebase Review: Master Issue List (updated 1 Nov 2026)

Scope: all 30 units in Codebase_for_claude.zip (about 24k lines excluding the test form): MemDB2 (core, API, buffering, indexing, journal, streaming), IndexedStore, BinaryTree, CoWTree, Reffed, Trackables, RWWLock/TinyLock/StripedLock/SRWLock, DLList/DLThreadQueue, Parallelizer, BufferedFileStream, NullStream, CRC32, PComp, PatchFreeMem. Line numbers refer to the source files. Release builds have **assertions disabled** (DCC_AssertionsAtRuntime=false), so every Assert-only check below is absent in production. 'Latent' means the defect exists but no current MemDB2 code path reaches it.

Category	High	Medium	Low / Low-Med	Total open	Resolved
1. Bugs	1	9	15	25	4
2. Optimisations	2	4	8	14	0
3. Not worth fixing (already handled)	-	-	-	17	-

What changed since the first review: stream I/O moved to ReadBuffer/WriteBuffer throughout; failed flushes now raise; journal and checkpoint files are written as *_partial, flushed, then renamed; NullStream.Write returns Count. IDs are kept from the first report; N-numbered items are new (N1/N2 come from the temp-file scheme, N3-N5 from the cached-stream follow-up).

Fix next: B2 (InitDB retry hang), N2 (non-durable rename can lose a flushed commit), N1 (temp-only directory blocks startup), N3 (stale read cache returns wrong data), B4/B5, then O1/O2.

1. Bugs (open, partly fixed, new)

ID	Severity	Status	Location	Problem	Suggested fix
B2	High	Open	MemDB2Journal.pas 577 (Initialise) + MemDB2.pas InitDB	Retrying InitDB after a failed initial load hangs forever: the journal thread has exited (FState=jscFinished) but FJournalThread is still assigned, so no new thread starts and InitDB waits on FInitWait (INFINITE).	In Initialise: if Assigned(FJournalThread) and FJournalThread.Finished then WaitFor + Free + nil before recreating.
N1	Medium	New	MemDB2Journal.pas 790-811 (GetListAndInitialSeqs)	If the directory holds only temp files (e.g. power loss during the very first checkpoint leaves 1.dbinit_partial), RList.Count is not 0, no final .dbinit is found and load fails with 'Couldn't find sequentially numbered db files'. Startup is blocked until the temp is deleted by hand.	Treat 'no final files' (ignoring temps) as the empty-DB case; keep temps in the list so cleanup removes them.
N2	Medium	New	MemDB2Journal.pas 1038, 1124 (RenameFile after flush)	The rename is not made durable (MoveFile without MOVEFILE_WRITE_THROUGH; no directory fsync on POSIX). After power loss a flushed amFlushBuffers file can reappear as *_partial and is then deleted by startup cleanup, losing a commit reported durable. A failed rename also still releases the transactions as if written.	Windows: MoveFileEx(..., MOVEFILE_WRITE_THROUGH). POSIX: fsync the directory after rename().
N3	Medium	New	BufferedFileStream.pas ~389-405 (TReadOnlyCachedFileStream.Read)	On a failed cache refill FCacheStart is moved but FCacheEnd keeps its old value. A later Read can then treat the stale range as valid and return bytes from the wrong file offset, with no error.	Set FCacheEnd := FCacheStart before refilling (invalidate the window).
B3	Medium	Partly fixed	MemDB2Journal.pas PerformCheckpoint / GetListAndInitialSeqs	FIXED: the checkpoint file is now flushed before rename. REMAINING: the loader always picks the newest .dbinit with no fallback, and the rename durability issue (N2) applies here too.	Write-through rename; optionally fall back to the previous .dbinit if the newest fails to replay.
B4	Medium	Open	MemDB2Journal.pas (CleanupUnusedFiles only from PerformInitialLoad)	Superseded journal files are only deleted at start-up; runtime checkpoints add a full DB copy each time and never reclaim older files, so disk use grows without bound during uptime.	After a checkpoint is durably renamed, delete files with seq < checkpoint seq.
B5	Medium	Open	MemDB2Journal.pas 322-335, 470-534	Shutdown use-after-free race: DoJournalFinished sets FreeOnTerminate and nils FJournalThread, then OnJournalFinished unblocks StopDB. The thread still runs Action.Free and re-reads Self.FState, and the destructor no longer waits.	Leave the loop right after jatLastAction via a local flag, or keep the thread ref and WaitFor in the destructor.
B7	Medium	Open	IndexedStore.pas 613, 797, 1182 and _AddIndex	Rollback runs only for EOutOfMemory. Any other exception from CompareItems (e.g. CompareFields type mismatch in lookaside nodes) leaves an item partly indexed with FCount not incremented, or a half-built index registered; on the async path the index is then freed under live links.	Catch Exception (not just EOutOfMemory) for rollback; re-raise.
B8	Medium	Open	RWWLock.pas 264	Batch end uses '=' (AdmittedContended = FRatios[Reason]). Lowering a ratio via SetRatio mid-batch means the batch never ends, starving other modes.	Use >=.
B9	Medium	Open	BufferedFileStream.pas 252 (SetSize)	Seek only updates FPosition, so SetEndOfFile acts at whatever the OS pointer is; the write cache isn't flushed and FFileSize isn't updated. Latent (not called by MemDB2).	FlushCache; FileSeek(NewSize); SetEndOfFile; update FFileSize and invalidate the read cache.
B10	Low/Med	Open	IndexedStore.pas 1021 (PerformAsyncActions)	If ExecParallel raises an exception outside the local filter, the cleanup tail never runs and ForceClearAsyncFlags then exposes dangling FIndexLink pointers. Async path not used by MemDB2.	try/finally around ExecParallel that always runs the tail.
B11	Low/Med	Open	IndexedStore.pas 1657 (DeleteChildren)	With an async-pending index, RemoveItem returns rvAsyncInProgress without removing, so the loop never ends and Obj.Free runs repeatedly on the same object.	Check RemoveItem result; break/raise.
B6	Low	Partly fixed	MemDB2Streamable.pas; BufferedFileStream.pas Read	FIXED: all stream I/O now uses ReadBuffer/WriteBuffer (or checked Read/Write), so short reads/writes raise. REMAINING: string/list/field-name lengths are not validated (corrupt length gives EOutOfMemory, not a clean error); cached-stream read errors set FLastError but are never surfaced, so you get a generic EReadError instead of the OS reason.	Validate lengths against Stream.Size - Position; call HandleError when FileRead returns -1.
B13	Low	Partly fixed	MemDB2Streamable.pas ToStream/RecFromStream (blob)	FIXED: blobs of 2 GiB or more now fail loudly (EWriteError/EReadError, or ERangeError in debug) instead of silently truncating. REMAINING: no up-front limit in WriteField, so the error appears at commit with a generic message; expect signed/unsigned compare warnings.	Reject size > MaxInt in TMemAPITableData.WriteField with a clear message.

ID	Severity	Status	Location	Problem	Suggested fix
N4	Low	New	BufferedFileStream.pas ~377-383 (Read, cache disabled)	A negative Count is passed straight to FileRead(Count: Longword), becoming a huge value: ReadFile can overrun the caller's buffer.	if Count <= 0 then Exit(0).
N5	Low	New	BufferedFileStream.pas 287-295 (base Read/Write)	Misuse (WriteBuffer on a read-only stream etc.) raises EAssertionFailed rather than an EStreamError descendant, so stream-error handlers miss it and it looks like a code bug.	Raise EStreamError/EReadError/EWriteError.
B12	Low	Open	RWWLock.pas 234 / 412 vs Destroy	Last thread signals FDestroyed inside FCrit then calls FCrit.Release while the destructor may already be freeing FCrit.	Signal after leaving FCrit, or Acquire/Release FCrit in the destructor before freeing.
B14	Low	Open	MemDB2Streamable.pas 860-869, 484-500	FromStream frees blob Data on error without nil-ing it (latent double free); RecFromStream tests Rec.Data, which overlaps integer fields; list FromStream leaks NewItem on failure.	Nil after FreeMem; Release NewItem in an except block.
B15	Low	Open	IndexedStore.pas 1895 (IndexInfoByOrdinal)	Checks Assigned(Tag) on the var output before it is populated; FAllowNullTags has no setter.	Drop the pre-check or check the result.
B16	Low	Open	Trackables.pas 106	Under USE_TRACKABLES, TTrackedMemoryStream derives from TTrackProxy instead of TMemoryStream. Currently unused.	class(TMemoryStream).
B18	Low	Open	MemDB2.pas 1126 (InitDB)	Compares un-normalised RootLocation with BaseDirectory (trailing slash added), so a second InitDB with the same path and no slash raises S_REINIT_DIFF_LOCATION.	AppendTrailingDirSlash(RootLocation) at the top of InitDB.
B19	Low	Open	MemDB2Journal.pas 550	Destroying a TMemDB that was never initialised trips Assert(FState = jscFinished) (state is jscCreated) in debug builds.	Accept jscCreated.
B20	Low	Partly fixed	MemDB2Journal.pas 705-740 (GetInitialFileList)	FIXED: dead LastFileBlank code removed. REMAINING: the early exit skips FindClose and leaks DBInfo objects; StrToInt64 accepts negative/hex names.	try/finally FindClose + FreeRList; validate names.
B21	Low	Open	MemDB2Buffered.pas 7723; Indexing TakeNodesFrom	Only the node cache is pre-sized, so the pin cache allocates mid-consolidation; out of memory there leaves pointers in two caches (double release).	Pre-size the pin cache; move then clear atomically.
B22	Low	Open	Reffed.pas 293, 328	GetExtraInfoText appends to an uninitialised Result; CloneByRef leaks the new list and refs if AddRef raises.	Initialise Result; try/except release in CloneByRef.

Resolved since the first review

ID	Severity	Was	Now
B1	High	Failed FlushFileBuffers/fsync was Assert-only, so ignored in release builds.	Now raises EMemDBException (S_FLUSH_FAILED) in both incremental writes and checkpoints.
B17	Low	TNullStream.Write returned 0, so WriteBuffer/CopyFrom raised.	Returns Count.
D1	High	Introduced in your interim diff: temp files still consumed a sequence number, so the restart after a crash recovery would fail to load.	Inc(FinalSeq) now only counts final files.
D2	Low	PerformMultiTransactionWrite/PerformCheckpoint freed an unassigned OutputStream in finally if file creation failed.	Creation moved outside the inner try/finally.

2. Optimisations

ID	Impact	Status	Location	Issue	Suggestion
O1	High	Open	Parallelizer.pas; MemDB2Buffered PreCommit/Commit/Rollback/index builds	ExecParallel creates one OS thread per task per call, and MemDB2 queues one task per entity per phase. Hundreds of tables means hundreds of thread creations per commit phase; a CreateThread failure in ccpData forces the DB into its error state.	Persistent pool of MaxParallel workers; run inline for a single task.
O2	High	Open	MemDB2Journal.pas PerformMultiTransactionWrite	Every write batch (every amFlushBuffers commit) is still a new file, now with an extra rename each. That means millions of files and slow start-up.	Rolling segment file with length + CRC32 framing per record.
O3	Medium	Open	MemDB2.pas Checkpoint + Journal PerformCheckpoint	Full DB serialised to a temp file, then copied to the .dbinit, doubling I/O and disk use.	Serialise straight to the *_partial file and rename (the temp+rename scheme now makes this easy).
O4	Medium	Open	MemDB2BufBase.pas 1070 (RequestChangeInternal)	DeepClone under the per-object TTinyLock spinlock.	Clone outside the lock; re-lock and verify Cur unchanged.
O5	Medium	Open	MemDB2Api.pas 1695 (API_DataFindEdgeByIndex)	O(k) walk across duplicates with two format checks per step.	Lower/upper-bound search in CoWTree, giving O(log n).
O6	Medium	Open	MemDB2Journal.pas 470	Waits up to 5 s between actions even when the queue is non-empty (no high-priority pending).	Only wait when the queue is empty.
O7	Low/Med	Open	MemDB2Buffered.pas 3801	Linear skip of deleted rows while holding FMasterRowLock (acknowledged TODO).	Bounded batches / drop and re-take the lock.
O8	Low/Med	Open	StripedLock.pas	Per-row heap TStripedLock object; 65,536 critical sections at unit init.	Inline the key; fewer padded stripes; lazy init.
O9	Low	Open	BufferedFileStream.pas	Large requests double-copied through the cache; write stream's read side uses a 16 KB cache.	Bypass for large requests; pass FILE_CACHE_SIZE.
O10	Low	Open	MemDB2Buffered.pas 1109	ParallelAddHandler grows five arrays by one each call (quadratic).	Pre-size.
O11	Low	Open	MemDB2Buffered.pas 7809	EntitiesByName linear scan with pins on every lookup.	Per-transaction name map.
O12	Low	Open	MemDB2Misc.pas 768	NeverSleepOnMMThreadContention forced globally.	Make it opt-in.
O13	Low	Open	TinyLock.pas	No backoff; Sleep(0) priority-inversion risk.	Exponential backoff / SwitchToThread.
O14	Low	Open	CRC32.pas / StripedLock	Byte-wise CRC to hash random GUIDs.	XOR-fold; slice-by-8 if CRC framing adopted.

3. Not worth fixing: already covered by error handling, rollback or exceptions

ID	Area	Why it's acceptable as-is
S1	MemDB2Api UserCommitCycle / UserRollbackCycle	Fail-stop design: pre-commit failures roll back index Next under the commit lock; anything after pre-commit, or a journalling failure, sets the DB error flag, which is checked synchronously under CommitLock, so memory and journal can't silently diverge. Rollback swallows the exception but flags the DB.
S2	MemDB2Journal AmalgamateTransactions / PerformMultiTransactionWrite	Every path, including exceptions, releases transactions, signals FlushFinishedEvent and balances HighPriorityDone; errors move the journal to its error state rather than killing the thread.
S3	TWriteCachedFileStream destructor flush failure	The exception propagates out of Free into PerformMultiTransactionWrite's handler and becomes DoJournalError; the handle is still closed in finally. Only the object memory leaks.
S4	BufferedFileStream constructor failure	CreateHandle raises before FHandle is assigned, FreeMem(nil) is safe, and FlushCache guards Assigned(FCache), so destruction of a partly-built object is clean.
S5	Parallelizer thread-creation failure mid-loop	finally frees (and joins) the threads already started; unstarted slots are nil. Re-raised exceptions keep the message but lose the subclass (e.g. EDivByZero becomes EIntError), which is acceptable.
S6	CoWTree insert/delete/rebalance	Exception-safe path copying: temps are released on exception and the 'consumed' flag disambiguates ownership of the supplied node. Release on nil is a no-op.
S7	TReffed TryAddRef / TryReleaseInt	CAS loop refuses to resurrect a zero refcount, so teardown races are detected and turned into an exception rather than a use-after-free.
S8	RemoveTransaction AsyncError rollback (goto txion_remove)	Skips the rollback cycle (pins leak), but only once the DB is already in its error state and must be restarted. Acknowledged TODO.
S9	Truncated final journal file	Now largely handled: writes go to *_partial files and are renamed only when complete, and startup ignores and deletes temps. Remaining gaps are N1/N2.
S10	GetAPIObject / PutAPIObjectLocal AddRef/Release window	Runs outside the lock, but entity proxies are also held by entity lists, so the refcount can't reach zero in that window (documented).
S11	PatchFreeMem	Only on XE4 Win64; byte-compares before patching, restores page protection in finally; returns false (ignored) on any mismatch.
S12	TMemDBDatabasePersistent / TMemDBTablePersistent release-mode destructors	Brute-force 'while TryRelease' loops run only at teardown with no clients; CHECK_TEARDOWN_REFS builds do the proper cleardown transaction.
S13	OOM-only leaks (StartTransaction list add, API_CreateTable/CreateIndex, GetAPIObject, CloneByRef)	Reachable only after allocation failure, where the process state is already compromised; documented as 'very unlikely'.
S14	Checkpoint holds CommitLock for the full serialise	Deliberate simplicity/safety trade-off, documented in source; commits block but stay consistent.
S15	TDLThreadQueue FDebugLocked shared flag	Debug-only check that another thread's lock can satisfy; no effect on release behaviour.
S16	TBinTree delete-by-CopyFrom node identity swap	Handled correctly for IndexedStore via TIndexNode.CopyFrom / IndexCopiedFrom re-linking; Trackables only uses internal items.
S17	FK pre-commit renamed table/index lookups returning nil	Nil results are caught by the 'same object' / AbsIdx checks, which raise before any dereference.